

Kubernetes

Namespaces

1. What is a Namespace?

In Kubernetes, a **namespace** is like a **virtual cluster within your cluster**. It helps you **organize and isolate resources** (pods, services, deployments, etc.) inside the same physical cluster.

Think of it like apartments in a building:

- The building is your cluster.
- Each apartment is a namespace.
- People in one apartment don't interfere with people in another apartment unless you allow them.

Use cases for namespaces:

1. **Environment separation** – dev, staging, prod.
2. **Team isolation** – multiple teams working in the same cluster.
3. **Resource management** – apply resource quotas per namespace.
4. **Security boundaries** – RBAC (Role-Based Access Control) can be set per namespace.

2. Default Namespaces in Kubernetes

Every Kubernetes cluster comes with some default namespaces:

Namespace	Purpose
default	For resources not assigned to any namespace
kube-system	For system components like kube-dns, kube-proxy

Namespace	Purpose
kube-public	Read-only namespace for public info, rarely used
kube-node-lease	Keeps track of node heartbeat

3. Commands to Work with Namespaces

List all namespaces

```
kubectl get namespaces
```

or

```
kubectl get ns
```

Create a namespace

```
kubectl create namespace dev
```

Delete a namespace

```
kubectl delete namespace dev
```

Caution: Deleting a namespace deletes **all resources inside it**.

Use a namespace (temporary for a command)

```
kubectl get pods --namespace=dev
```

Set a default namespace for your session

```
kubectl config set-context --current --namespace=dev
```

4. Assigning Resources to a Namespace

In your YAML manifest, you can assign a namespace like this:

```
apiVersion: v1
```

```
kind: Pod
```

```
metadata:
```

```
name: my-app
namespace: dev # <-- assign to dev namespace
spec:
  containers:
  - name: nginx
    image: nginx
```

If you **don't specify**, it will go into the default namespace.

5. Why Namespaces Matter Practically

- If you `kubectl get pods` without `-n`, you see pods only in default.
- Multiple teams can use the same cluster safely.
- Resource quotas, limits, and RBAC policies are applied **per namespace**, so your dev team won't accidentally exhaust resources for prod.

✓ Summary:

- Namespaces = logical isolation inside a cluster.
- Default namespace = default.
- Assign resources via YAML or `kubectl --namespace`.
- Useful for isolation, organization, and security.

1. ResourceQuota

A **ResourceQuota** is a Kubernetes object that **limits the total resource consumption in a namespace**. It can limit:

- CPU
- Memory
- Number of Pods, Services, PersistentVolumeClaims
- Storage

Here's an example:

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: dev-quota
  namespace: dev # <-- this quota is applied to the 'dev' namespace
spec:
  hard:
    pods: "10" # max 10 pods in dev
    requests.cpu: "4" # total CPU requests ≤ 4 cores
    requests.memory: 8Gi # total memory requests ≤ 8Gi
    limits.cpu: "8" # total CPU limits ≤ 8 cores
    limits.memory: 16Gi # total memory limits ≤ 16Gi
```

What happens:

- If the dev team tries to create the 11th pod, it will **fail**.
- If the total CPU/memory requested by pods exceeds the quota, new pods are rejected.

2. LimitRange

While **ResourceQuota** controls the **total usage** for a namespace, **LimitRange** controls **per-pod or per-container defaults and limits**.

Example:

```
apiVersion: v1
kind: LimitRange
metadata:
  name: dev-limits
  namespace: dev
spec:
  limits:
    - type: Container
      max:
        cpu: "2"
        memory: 4Gi
      min:
        cpu: "100m"
```

memory: 128Mi
default:
cpu: "500m"
memory: 512Mi
defaultRequest:
cpu: "250m"
memory: 256Mi

Explanation:

- Every container in dev namespace without explicit CPU/memory will get defaultRequest and default automatically.
- min prevents tiny pods, max prevents oversized pods.

3. How they work together

1. **LimitRange** → per pod/container limits/defaults in a namespace.
2. **ResourceQuota** → overall limit for the namespace.

Example scenario:

- dev namespace has ResourceQuota pods=10 and requests.cpu=4.
- LimitRange ensures no container asks for more than 2 CPU.
- You can have 5 pods with 0.8 CPU each, total = 4 CPU → allowed.
- Any pod exceeding 2 CPU individually → blocked by LimitRange.

4. Applying ResourceQuota and LimitRange

kubectl apply -f dev-quota.yaml
kubectl apply -f dev-limits.yaml

Then, if you try:

apiVersion: v1
kind: Pod

```
metadata:
  name: big-pod
  namespace: dev
spec:
  containers:
  - name: nginx
    image: nginx
    resources:
      requests:
        cpu: "3" # fails if quota + limit exceeded
        memory: 5Gi
      limits:
        cpu: "5" # fails if limitrange exceeded
        memory: 10Gi
```

Kubernetes will reject this pod if it violates **either the namespace quota or the limit range**.

✓ Summary:

- Namespaces **don't store resources** directly.
- Use **ResourceQuota** for **total namespace allocation**.
- Use **LimitRange** for **per-pod/container limits**.
- Together, they ensure fair usage among teams and environments.

Ingress

Imagine your cluster is a big apartment building:

- Each apartment (pod) has a room (container) inside.
- Inside the room, there's a toy (app) listening on a certain port (like 3000).
- People outside the building want to play with your toy. You need doors and keys to connect.

Now the ports:

1. Container Port (inside the pod)

- This is the **port your app listens on inside the container**.
- Example: nginx listens on port 80 inside the container.
- Only the pod itself knows this port. **Outside world cannot access it directly.**

Think of it like: **the toy inside the room**. Only the kid in the room can play with it.

containers:

- name: nginx

image: nginx

ports:

- containerPort: 80 # toy listens here

2. Pod IP / Internal cluster communication

- Every pod gets its **own IP inside the cluster**.
- Other pods can talk to it using **containerPort + pod IP**.

Think: **neighbors in the building can knock on your door**.

3. Service Port / TargetPort / NodePort

When you want to let people outside the pod play with the toy, you need a service:

```
apiVersion: v1
kind: Service
metadata:
name: my-service
spec:
type: NodePort
selector:
app: my-app
ports:
- port: 80 # Service port (inside cluster)
targetPort: 80 # Pod/container port
nodePort: 30007 # External port on the node
```

Let's break these:

Name	Where it lives	What it does	Analogy
targetPort	Pod/container	Actual port of your app	The toy inside the room (containerPort)
port	Service inside cluster	Port for other pods to talk to your service	The door inside the apartment hallway (other pods knock here)
nodePort	Node	Opens a door from outside the building	A special door on the building's wall (external world can use this)

4. ClusterIP (default)

- The service gets a **virtual IP inside the cluster**.
- Only other pods can talk to it.
- No nodePort, no external access.

Analogy: **a hallway door inside the building** — only neighbors can knock.

5. LoadBalancer

- Mostly in cloud clusters (AWS, GCP, Azure).
- Opens a **real external IP** and routes traffic to the service.
- Can be seen as a **big mailbox / front gate** that connects outside world to your NodePort doors.

Analogy: a **building gate with a receptionist** that routes visitors to the right apartment.

6. Ingress

- An Ingress is like a **receptionist who decides exactly which room to send visitors to**, based on **URL or hostname**.
- It doesn't replace NodePort or LoadBalancer; it **sits on top of them**.

Analogy:

- NodePort/LoadBalancer = building gate.
- Ingress = receptionist → "Ah, you want the booking-service? Go to 2nd floor, room 5."

1. What is Ingress?

An **Ingress** in Kubernetes is a **layer that manages external access to services inside your cluster**, typically HTTP/HTTPS.

Think of it as a **smart entry gate** to your cluster:

- You have multiple services (like frontend, backend, booking-service).
- Instead of exposing each service via a NodePort or LoadBalancer, you use **Ingress** to route traffic based on **hostnames** or **URLs**.

It's like a **reverse proxy** inside your cluster.

2. Why not just NodePort or LoadBalancer?

- **NodePort** – exposes a service on a port of each node. Hard to manage multiple services.
- **LoadBalancer** – works in cloud environments, but each service gets its own external IP (can be costly).
- **Ingress** – single entry point, can route to many services, supports TLS (HTTPS), path-based routing, host-based routing.

3. Ingress Components

1. **Ingress Resource** – YAML object that defines the rules (host/path → service).
2. **Ingress Controller** – the actual component that implements the rules (like **NGINX Ingress Controller**, **Traefik**, **HAProxy**).

Note: You must install an **Ingress Controller** in your cluster; the Ingress resource alone does nothing.

4. Example: Basic Ingress

Suppose you have two services: frontend-service and booking-service. Here's how you can route traffic:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: my-ingress
  namespace: dev
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  rules:
    - host: myapp.local
      http:
        paths:
          - path: /booking
```

```
pathType: Prefix
backend:
  service:
    name: booking-service
    port:
      number: 80
- path: /frontend
pathType: Prefix
backend:
  service:
    name: frontend-service
    port:
      number: 80
```

Explanation:

- Requests to `http://myapp.local/booking` → go to booking-service.
 - Requests to `http://myapp.local/frontend` → go to frontend-service.
 - All traffic goes through **Ingress Controller**, which handles routing.
-

5. Adding HTTPS with TLS

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: my-ingress
  namespace: dev
spec:
  tls:
    - hosts:
        - myapp.local
      secretName: myapp-tls
  rules:
    - host: myapp.local
      http:
        paths:
          - path: /
```

```
pathType: Prefix
backend:
  service:
    name: frontend-service
    port:
      number: 80
```

You need a **TLS secret** (certificate + key) created in the namespace.

6. Commands to check Ingress

```
kubectl get ingress -n dev
```

```
kubectl describe ingress my-ingress -n dev
```

7. Summary

- **Ingress** = HTTP/HTTPS routing to services.
- Requires an **Ingress Controller** to function.
- Supports **path-based** and **host-based routing**, TLS.
- Simplifies exposing multiple services under one domain/IP.

What is an Ingress Controller?

- An **Ingress resource** is just a **set of rules** in Kubernetes.
- But **Kubernetes itself doesn't know how to route traffic** to your services based on those rules.
- That's where an **Ingress Controller** comes in — it **listens to the Ingress resources** and implements the routing.

Analogy:

- **Ingress resource** = instructions for the receptionist.
- **Ingress Controller** = the receptionist who actually opens doors and sends visitors to the correct room.

Without an Ingress Controller, creating an Ingress resource does **nothing**.

2. NGINX Ingress Controller

- One of the **most common controllers**.
- Uses **NGINX** as the actual reverse proxy to route traffic.
- Supports:
 - Path-based routing (/booking, /frontend)
 - Host-based routing (myapp.local, api.myapp.local)
 - SSL/TLS termination
 - Rewrite rules and annotations

How it works:

1. You deploy NGINX Ingress Controller as a pod in your cluster.
2. It watches the API server for any Ingress objects.
3. When it sees /booking → booking-service, it configures NGINX to route traffic automatically.

Pros:

- Battle-tested, widely used
- Lots of annotations for custom behavior
- Works well with cloud and on-prem

Cons:

- Slightly heavier than Traefik
- Config can be verbose

3. Traefik Ingress Controller

- Another popular controller, lightweight and modern.
- Written in **Go**, has **dynamic configuration** abilities.
- Supports:
 - HTTP/HTTPS routing

- Automatic Let's Encrypt SSL
- Web UI dashboard to see routes
- Can handle TCP/UDP in addition to HTTP

How it works:

- Traefik pod watches Ingress resources (like NGINX).
- Generates routing rules dynamically.
- Can integrate with Kubernetes CRDs (IngressRoute) for advanced routing.

Pros:

- Lightweight
- Auto SSL with Let's Encrypt
- Easy to visualize routes
- Dynamic config without restarts

Cons:

- Less mature than NGINX in some advanced Kubernetes scenarios

Browser -> Ingress Controller (NGINX/Traefik) -> Ingress Rules -> Service -> Pod

- Browser hits myapp.local/booking
- Ingress Controller sees rule /booking -> booking-service
- Controller sends request to the service
- Service routes to the correct pod

✓ Summary:

- **Ingress resource** = rules
- **Ingress Controller** = the engine that enforces rules
- **NGINX** = heavy, mature, flexible
- **Traefik** = lightweight, modern, auto SSL

Implementation

Step 0 — Prerequisites

- Minikube installed
- kubectl installed and pointing to your cluster (minikube start)
- Optional: Docker if building custom images

Check:

`kubectl get nodes`

You should see your minikube node ready.

Step 1 — Start Minikube with Ingress Enabled

`minikube start`

`minikube addons enable ingress`

What this does:

- Starts a local Kubernetes cluster
- Enables **NGINX Ingress Controller** automatically as a pod inside kube-system
- Now your cluster can handle Ingress rules

Check:

`kubectl get pods -n kube-system | grep ingress`

You should see something like nginx-ingress-controller-xxxx running.

Step 2 — Create Namespaces

We'll isolate dev and prod:

```
kubectl create namespace dev  
kubectl create namespace prod
```

What this does:

- Separates resources for dev and prod
- Helps us avoid conflicts

Step 3 — Create Backend Deployment

We'll create a simple backend (Node.js or simple NGINX JSON API). Example using NGINX returning JSON:

What this does:

- Creates 2 replicas of backend pod
- Pods run container that responds with "Hello from Backend" on port 5678

Apply it:

```
kubectl apply -f backend-deploy.yaml
```

Check pods:

```
kubectl get pods -n dev
```

Step 4 — Create Backend Service

What this does:

- Creates a **stable network endpoint** for backend pods
- Other pods (or Ingress) can talk to backend-service:80
- ClusterIP = internal-only access

Apply it:

```
kubectl apply -f backend-service.yaml  
kubectl get svc -n dev
```

Step 5 — Create Frontend Deployment

Simple frontend (Node.js or http-echo again):

What this does:

- Creates 2 frontend pods that respond on port 5678

Apply it:

```
kubectl apply -f frontend-deploy.yaml
```

Step 6 — Create Frontend Service

What this does:

- Provides a **stable endpoint for frontend pods**
- Other pods or Ingress can route traffic to it

Apply it:

```
kubectl apply -f frontend-service.yaml
```

```
kubectl get svc -n dev
```

Step 7 — Create Ingress Resource

```
apiVersion: networking.k8s.io/v1
```

```
kind: Ingress
```

```
metadata:
```

```
  name: dev-ingress
```

```
  namespace: dev
```

```
  annotations:
```

```
    nginx.ingress.kubernetes.io/rewrite-target: /
```

```
spec:
```

```
  rules:
```

```
    - host: myapp.local
```

http:

paths:

- path: /backend

pathType: Prefix

backend:

service:

name: backend-service

port:

number: 80

- path: /frontend

pathType: Prefix

backend:

service:

name: frontend-service

port:

number: 80

What this does:

- Defines **external routing rules**
- Requests to `http://myapp.local/backend` → backend-service
- Requests to `http://myapp.local/frontend` → frontend-service
- `rewrite-target: /` ensures the services receive requests at root path

Apply:

```
kubectl apply -f dev-ingress.yaml
```

```
kubectl get ingress -n dev
```

Step 8 — Test It Locally

1. Get minikube IP:

`minikube ip`

2. Edit your `/etc/hosts` (on Linux/Mac) and add:

`<minikube-ip> myapp.local`

Step 9 — Recap of Each Component

Component	What it does
Namespace	Isolates dev/prod resources
Deployment	Creates replicas of pods (frontend/backend)
Pod	Runs container with your app
ContainerPort	Port your app listens on inside pod
Service	Provides stable network endpoint for pods (ClusterIP)
Ingress	Routes external requests to services
NGINX Ingress Controller	Watches Ingress rules and handles the actual routing

1. What is a ConfigMap?

A **ConfigMap** is a Kubernetes object that **stores configuration data** separately from your application code.

- Think of it as a **settings file for your app**, but managed by Kubernetes.
- Allows you to **change configuration without rebuilding the container image**.

Analogy:

- Your pod = robot
- Robot has instructions hardcoded (code) and settings (ConfigMap)
- ConfigMap = external notebook with settings your robot reads

2. Why use ConfigMaps?

- Keep **code and config separate** (best practice)
- **Update configuration without redeploying** (just reload pods)
- Share config across multiple pods
- Works well for environment variables, config files, or command-line args

3. Types of ConfigMap usage

1. **As environment variables**
2. **As command-line arguments**
3. **Mounted as files inside a pod**

4. Example 1 — ConfigMap as environment variables

apiVersion: v1

kind: ConfigMap

metadata:

name: app-config

```
namespace: dev
data:
  APP_MODE: "development"
  APP_COLOR: "blue"
```

What this does:

- Creates a ConfigMap called app-config in dev namespace
- Stores key-value pairs: APP_MODE=development and APP_COLOR=blue

Use it in a Deployment:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
  namespace: dev
spec:
  replicas: 1
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
        - name: my-app
          image: nginx
          envFrom:
            - configMapRef:
                name: app-config
```

✓ Now the container has **environment variables APP_MODE and APP_COLOR** injected from the ConfigMap.

5. Example 2 — ConfigMap mounted as a file

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config-file
  namespace: dev
data:
  settings.txt: |
    APP_MODE=development
    APP_COLOR=blue
```

Use it in a pod:

```
apiVersion: v1
kind: Pod
metadata:
  name: my-app
  namespace: dev
spec:
  containers:
    - name: my-app
      image: nginx
      volumeMounts:
        - name: config-volume
          mountPath: /etc/config/settings.txt
          subPath: settings.txt
  volumes:
    - name: config-volume
      configMap:
        name: app-config-file
```

What this does:

- Mounts the ConfigMap as a **file inside the pod**
- App can read config from `/etc/config/settings.txt`

6. Updating ConfigMap

```
kubectl edit configmap app-config -n dev
```

Or

```
kubectl apply -f updated-config.yaml
```

- Pods **don't automatically pick up changes** unless:
 - They are restarted
 - Or if the ConfigMap is mounted as a volume, some apps watch the file for changes

7. Commands

```
kubectl get configmap -n dev
```

```
kubectl describe configmap app-config -n dev
```

```
kubectl delete configmap app-config -n dev
```

Summary

- **ConfigMap** = store non-secret config (env vars, files) outside pods
- **Inject** via: environment variables, command args, or file mounts
- **Separate config from code** → easier updates & management

Secrets

1. What is a Secret?

A **Secret** is a Kubernetes object used to **store sensitive information** separately from your code.

- Think of it as a **locked safe for your passwords**.
- Unlike ConfigMaps, Secrets are **base64-encoded** (not encrypted by default, but safer than plain text).
- Pods can access Secrets **as environment variables, mounted files, or through Kubernetes APIs**.

2. Why use Secrets?

- Don't hardcode sensitive data in your container images
- Share credentials safely across multiple pods
- Enable easy rotation of secrets without rebuilding images

3. Create a Secret

Method 1 — From literal values

```
kubectl create secret generic db-secret \
  --from-literal=username=admin \
  --from-literal=password=SuperSecret123 \
  -n dev
```

- Creates a secret called db-secret in dev namespace
- Stores username and password

Check:

```
kubectl get secrets -n dev
kubectl describe secret db-secret -n dev
```

The values are **base64-encoded** when you look at the raw YAML.

Method 2 — From a file

```
echo -n 'SuperSecret123' > password.txt
```

```
kubectl create secret generic db-secret --from-file=password=password.txt -n dev
```

- You can store passwords, keys, or certificates in files and create a Secret from them

4. Using Secrets in Pods

As environment variables

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:
```

```
  name: my-app
```

```
  namespace: dev
```

```
spec:
```

```
  replicas: 1
```

```
  selector:
```

```
    matchLabels:
```

```
      app: my-app
```

```
  template:
```

```
    metadata:
```

```
      labels:
```

```
        app: my-app
```

```
    spec:
```

```
      containers:
```

```
        - name: my-app
```

```
          image: nginx
```

```
          env:
```

```
            - name: DB_USERNAME
```

```
              valueFrom:
```

```
                secretKeyRef:
```

```
                  name: db-secret
```

```
                  key: username
```

```
- name: DB_PASSWORD
  valueFrom:
    secretKeyRef:
      name: db-secret
      key: password
```

What this does:

- Injects secrets into the pod as **environment variables**
 - App can use DB_USERNAME and DB_PASSWORD without hardcoding them
-

As mounted files

```
apiVersion: v1
kind: Pod
metadata:
  name: secret-pod
  namespace: dev
spec:
  containers:
    - name: my-app
      image: nginx
      volumeMounts:
        - name: secret-volume
          mountPath: /etc/secrets
  volumes:
    - name: secret-volume
      secret:
        secretName: db-secret
```

What this does:

- Mounts secrets as files in /etc/secrets
 - File names = keys in Secret (username, password)
 - App can read them from disk
-

5. Commands

`kubectl get secrets -n dev`

`kubectl describe secret db-secret -n dev`

`kubectl delete secret db-secret -n dev`

6. Key Differences Between ConfigMap and Secret

Feature	ConfigMap	Secret
Purpose	Non-sensitive config	Sensitive info (passwords)
Storage	Plain text	Base64 encoded
Usage	Env vars, args, files	Env vars, files, API
Security	No encryption by default	Safer, can enable encryption at rest
Ideal For	Settings, flags, app config	Passwords, API keys, tokens

Summary

- **Secrets** = sensitive version of ConfigMaps
- Can be used as **env vars** or **files**
- Keep secrets out of code and images
- Can be rotated without rebuilding pods

1. TLS Certificates in a Nutshell

Before understanding certificate management, recall what a **TLS certificate** does:

1. **Encryption** – Scrambles data so no one can eavesdrop
2. **Authentication** – Confirms that “this website/service really is who it says it is”
3. **Integrity** – Ensures data isn’t tampered with during transit

Think of it as:

- A **lock + key** for your website traffic.
- Only people with the right key (browser + server) can read the data.

2. The Problem Without Management

Manually dealing with certificates has several pain points:

- You have to **request the certificate** from a CA (Certificate Authority) like Let’s Encrypt
- You have to **store it somewhere safe**
- You have to **inject it into your pods/services**
- Certificates **expire** (usually 90 days with Let’s Encrypt) → if you forget to renew, HTTPS breaks
- Scaling is painful: multiple services → multiple certificates → lots of manual work

3. What Certificate Management Solves

Certificate management (like **cert-manager** in Kubernetes) **automates all of this**. Here’s what it does step by step:

Step A — Certificate Request

- It asks a **Certificate Authority (CA)** to issue a certificate for your domain.

- Example: “Let’s Encrypt, please give me a valid certificate for myapp.local.”

Step B — Domain Validation

- The CA needs to check you **own the domain**.
- cert-manager automatically configures **temporary Ingress or DNS challenges** to prove domain ownership.

Step C — Certificate Storage

- Once issued, the certificate + private key is stored as a **Kubernetes Secret**.
- Apps/pods can mount the Secret automatically.

Step D — Automatic Renewal

- Certificates have a short lifespan (e.g., 90 days).
- cert-manager **monitors expiration** and automatically requests a new certificate **before the old one expires**.

Step E — Integration with Ingress

- Updates your Ingress to use the new certificate seamlessly

What Certificate Management Actually Does

1. **Requests certificates** from trusted authorities
2. **Validates your domain automatically**
3. **Stores certificates securely** as Kubernetes Secrets
4. **Automatically renews** certificates before expiration
5. **Integrates with Ingress** so your apps always serve HTTPS
6. Reduces human error and keeps **scalable, secure HTTPS** for multiple apps

1. Why Certificate Management?

- Ingress often exposes your apps to the internet.

- **TLS certificates** encrypt traffic between clients (browser) and your services.
- Manual certificate management is **error-prone**, so Kubernetes uses tools to **automate issuance, renewal, and storage**.

Analogy:

- Think of TLS as a **locked mailbox** for visitors.
- Certificate management ensures the **lock is valid and renewed automatically**, so nobody can peek in.

2. Methods for Managing Certificates in Kubernetes

A. Manual TLS with Secrets

1. Obtain a certificate and key (e.g., from OpenSSL or Let's Encrypt).
2. Create a Kubernetes Secret:

```
kubectl create secret tls myapp-tls \
--cert=fullchain.pem \
--key=privkey.pem \
-n dev
```

- Secret type `tls` stores **certificate + private key**
- Use this Secret in your Ingress:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: dev-ingress
  namespace: dev
spec:
  tls:
  - hosts:
    - myapp.local
    secretName: myapp-tls
```

```
rules:
- host: myapp.local
  http:
    paths:
    - path: /
      pathType: Prefix
    backend:
      service:
        name: frontend-service
        port:
          number: 80
```

✓ Browser requests to <https://myapp.local> now use the certificate in myapp-tls.

Limitations:

- Manual renewal required
- Error-prone for multiple services

B. Automated Certificate Management with cert-manager

- **cert-manager** is the standard Kubernetes tool to **automate TLS certificate issuance and renewal**.
- Works with **Let's Encrypt**, **self-signed certs**, or internal CAs.
- Can automatically create **Secrets** and update Ingress.

Step 1 — Install cert-manager

[kubectl apply -f https://github.com/cert-manager/cert-manager/releases/download/v1.14.4/cert-manager.yaml](https://github.com/cert-manager/cert-manager/releases/download/v1.14.4/cert-manager.yaml)

- Installs **CRDs + cert-manager controller pods**

Check pods:

[kubectl get pods -n cert-manager](#)

Step 2 — Create ClusterIssuer or Issuer

Issuer defines **how cert-manager gets certificates**. Example with Let's Encrypt (staging):

apiVersion: cert-manager.io/v1

kind: ClusterIssuer

metadata:

name: letsencrypt-staging

spec:

acme:

server: https://acme-staging-v02.api.letsencrypt.org/directory

email: your-email@example.com

privateKeySecretRef:

name: letsencrypt-staging

solvers:

- http01:

ingress:

class: nginx

- http01 solver uses **Ingress** to prove domain ownership
- Controller automatically requests certificate and stores it in a Secret

Step 3 — Annotate Ingress to request certificate

apiVersion: networking.k8s.io/v1

kind: Ingress

metadata:

name: dev-ingress

namespace: dev

annotations:

cert-manager.io/cluster-issuer: letsencrypt-staging

spec:

tls:

- hosts:

- myapp.local

secretName: myapp-tls

rules:

```
- host: myapp.local
http:
  paths:
    - path: /
      pathType: Prefix
      backend:
        service:
          name: frontend-service
          port:
            number: 80
```

✓ cert-manager automatically:

- Issues a certificate from Let's Encrypt
- Creates a **Secret** called myapp-tls
- Updates Ingress when the certificate is renewed

3. Summary

- **TLS/HTTPS** = encrypt traffic to your services
- Can use **manual TLS secrets** or **cert-manager** for automation
- **cert-manager** integrates with Ingress, automatically issues and renews certificates
- Best practice: **use cert-manager in production**

Volumes

1. Why do we need Volumes?

- Containers are **ephemeral** — when a pod is deleted, all its data disappears.
- Many apps (databases, logs, uploads) need **persistent storage**.

Analogy:

- Pod = disposable robot
- Container = robot's temporary memory (wiped when turned off)
- Volume = notebook or hard drive that sticks around even if the robot is replaced

2. Kubernetes Volumes

- A **Volume** attaches storage to a pod.
- Lives as long as the **pod exists** (data disappears if pod is deleted unless using persistent storage).
- Can be backed by: hostPath, emptyDir, ConfigMap, Secret, NFS, cloud storage, etc.

Example:

```
apiVersion: v1
kind: Pod
metadata:
  name: volume-demo
spec:
  containers:
    - name: app
      image: nginx
      volumeMounts:
        - name: my-volume
          mountPath: /usr/share/nginx/html
  volumes:
```

- name: my-volume
emptyDir: {}

What this does:

- emptyDir → temporary storage for the pod
- When pod is deleted, data is gone

3. Persistent Volumes (PV)

- A **PersistentVolume** is **cluster-level storage**.
- Can exist **independently of pods**.
- Backed by real storage: NFS, AWS EBS, GCP PD, Azure Disk, etc.
- Admin creates PVs, sets size, access mode (ReadWriteOnce/Many), storage class, etc.

Example:

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-demo
spec:
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  hostPath:
    path: /mnt/data
```

Explanation:

- 1Gi storage allocated
- Can be mounted **read-write by 1 pod at a time**
- ReclaimPolicy Retain → keeps data even if PV is released

4. Persistent Volume Claim (PVC)

- A **PVC** is like a **request for storage** by a pod.
- Pods don't directly know about PV; they just ask for **size + access mode**.
- Kubernetes **binds the PVC to a matching PV** automatically.

Example:

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc-demo
  namespace: dev
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 500Mi
```

What happens:

- Pod requests 500Mi storage
- Kubernetes finds a PV with enough capacity and compatible access mode
- PVC becomes the “bridge” between **pod → PV**

5. Using PVC in a Pod

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-with-pvc
  namespace: dev
spec:
  containers:
    - name: app
```

```
image: nginx
volumeMounts:
  - name: my-storage
    mountPath: /usr/share/nginx/html
volumes:
  - name: my-storage
    persistentVolumeClaim:
      claimName: pvc-demo
```

Explanation:

- Pod mounts PVC at /usr/share/nginx/html
- Data persists even if pod is deleted and recreated

6. Summary Table

Term	Scope	Lifespan	Purpose
Volume	Pod	Pod lifecycle	Temporary storage for pod/container
PersistentVolume (PV)	Cluster	Independent of pod	Actual physical storage allocated by admin
PersistentVolumeClaim (PVC)	Pod	Pod uses it	Pod's request for storage, bound to a PV

7. Analogy

- Pod = robot
- Container = robot's brain (memory wiped on death)
- Volume = notebook that robot can carry while alive
- PV = warehouse storage in the city (cluster-level, persistent)
- PVC = robot's request to the warehouse: "I need 500MB to store stuff"
- Pod → PVC → PV = robot gets warehouse space automatically

Resource Management

1. What is Resource Management?

In Kubernetes, **resource management** = **controlling how much CPU & Memory your containers use** so:

- One app doesn't **eat all resources**
- Cluster stays **stable**
- Apps get **fair share**

2. The Two Most Important Things

Every container can define:

resources:

requests:

limits:

A. Requests (Minimum Guarantee)

requests:

cpu: "200m"

memory: "256Mi"

👉 This means:

- "I need at least this much to run properly"

Kubernetes Scheduler uses this to:

- Decide **which node to place the pod on**

🧠 Think:

"Reserve this much space for me"

B. Limits (Maximum Cap)

limits:

cpu: "500m"

memory: "512Mi"

👉 This means:

- "Don't let me use more than this"

💡 Think:

"I can grow, but only up to this point"

3. CPU vs Memory Behavior (IMPORTANT ⚠️)

This is where most people mess up.

CPU (Soft Limit)

- If container exceeds CPU limit → it gets **throttled (slowed down)**
- It **does NOT crash**

💡 Analogy:

You're allowed to run fast, but if you exceed speed → system slows you down

Memory (Hard Limit)

- If container exceeds memory limit → **KILLED (OOMKilled)**
- Pod may restart

💡 Analogy:

You exceed memory → system says "OUT!" 💀

4. Example (Real YAML)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
  namespace: dev
spec:
  replicas: 2
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
        - name: app
          image: nginx
          resources:
            requests:
              cpu: "200m"
              memory: "256Mi"
            limits:
              cpu: "500m"
              memory: "512Mi"
```

5. What Happens Internally

Step 1 — Scheduling

- Scheduler checks:
“Which node has $\geq$ 200m CPU & 256Mi memory free?”

Step 2 — Running

- Container starts using resources

Step 3 — If it exceeds:

- CPU > 500m → throttled
- Memory > 512Mi → killed

6. Namespace-Level Control (You Already Saw This)

This connects to what you learned earlier 🙋

A. ResourceQuota (Total Limit per Namespace)

spec:

hard:

requests.cpu: "4"

requests.memory: 8Gi

👉 Total usage in namespace cannot exceed this

B. LimitRange (Per Container Rules)

limits:

- type: Container

max:

cpu: "2"

min:

cpu: "100m"

👉 Enforces:

- Minimum & maximum per container
- Default values

7. Why This Matters in Real Systems

Without resource management:

- ✗ One bad pod → crashes entire node
- ✗ Memory leak → kills everything
- ✗ No fairness → unstable cluster

With it:

- ✓ Stable system
- ✓ Predictable performance
- ✓ Better scaling

9. Pro-Level Insight (Important)

You might think:

“Why not set high limits always?”

Bad idea ✗

- Wastes cluster resources
- Reduces scheduling efficiency
- Can cause **node overcommit issues**

👉 Best practice:

- Set **realistic requests**
- Set **slightly higher limits**

10. Connect Everything (Big Picture)

Namespace



ResourceQuota (total limit)



LimitRange (per container rules)



Pod



Container (requests + limits)

Horizontal Pod Autoscaling

1. What is HPA?

HPA automatically increases or decreases the number of pods based on load.

👉 Instead of:

“I think I need 3 replicas”

You say:

“Keep CPU around 50%, adjust pods automatically”

2. Why HPA Exists

Problem without HPA:

- Fixed replicas → either:
 - ❌ Too few → app crashes under load
 - ❌ Too many → waste resources

HPA solves this by:

- ✅ Scaling **up** when load increases
- ✅ Scaling **down** when load decreases

3. How HPA Works (Core Logic)

HPA continuously checks:

Current CPU usage vs Target CPU usage

Then:

If usage > target → increase pods

If usage < target → decrease pods

Formula (simple version)

$\text{desiredReplicas} = \text{currentReplicas} \times (\text{currentCPU} / \text{targetCPU})$

Example:

- Current pods = 2
- Current CPU = 80%
- Target CPU = 50%

$2 \times (80 / 50) = 3.2 \rightarrow 3$ pods

👉 HPA scales to **3 pods**

4. Requirements (IMPORTANT ⚠)

HPA **won't work unless:**

✅ 1. Metrics Server installed

`kubectl top pods`

If this fails → HPA won't work.

For minikube:

`minikube addons enable metrics-server`

✅ 2. Resource requests must be set

HPA uses **CPU % based on requests**

requests:

`cpu: "200m"`

👉 Without this → HPA has no baseline → doesn't work

5. HPA YAML Example

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: my-app-hpa
  namespace: dev
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: my-app # target deployment
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 50
```

Explanation (Line by Line)

- scaleTargetRef → which deployment to scale
- minReplicas → never go below 2
- maxReplicas → never exceed 10
- averageUtilization: 50 → target CPU = 50%

6. What Happens in Real Flow

User traffic increases



Pods CPU increases



Metrics Server reports usage

↓
HPA calculates needed replicas
↓
Deployment scales pods
↓
Load gets distributed

7. Commands to Monitor HPA

```
kubectl get hpa -n dev  
kubectl describe hpa my-app-hpa -n dev  
kubectl top pods -n dev
```

8. Real Example (Test It)

Step 1 — Deploy app with resources

```
resources:  
  requests:  
    cpu: "100m"  
  limits:  
    cpu: "500m"
```

Step 2 — Create HPA

```
kubectl apply -f hpa.yaml
```

Step 3 — Generate load

```
kubectl run -i --tty load-generator --rm --image=busybox -- /bin/sh
```

Inside pod:

```
while true; do wget -q -O- http://your-service; done
```

👉 This increases CPU → HPA reacts

9. Important Behaviors (Advanced Insight)

Scaling is NOT instant

- Happens every ~15 seconds (default sync loop)
-

Cooldown period

- Prevents rapid up/down scaling (flapping)
-

Works best with stateless apps

- Scaling databases horizontally is harder

10. Common Mistakes (You should avoid)

- ✗ No resource requests → HPA breaks
- ✗ Metrics server not installed → no scaling
- ✗ Setting min=max → no scaling
- ✗ Too aggressive target (like 10%) → over-scaling

11. Mental Model

Think:

Deployment = how many pods NOW

HPA = adjusts that number automatically

12. Big Picture (Connect Everything)

User Traffic ↑



CPU Usage ↑



Metrics Server



HPA



Deployment replicas ↑



More Pods handle load

Node Autoscaling

1. What is Node Autoscaling?

Node Autoscaling = automatically adding/removing machines (nodes) in your cluster based on demand.

👉 HPA scales **pods**

👉 Node Autoscaler scales **infrastructure**

2. Why Do We Need It?

Imagine this situation:

- HPA says: “We need 10 pods”
- But your cluster has capacity for only 5 pods

❌ Remaining pods stay in **Pending state**

Node Autoscaler solves this:

Pods need space → No node available → Add new node → Pods scheduled

3. Types of Autoscaling (Big Picture)

Type	Scales What
HPA	Pods
VPA	Pod resources (CPU/memory)
Cluster Autoscaler	Nodes

4. How Cluster Autoscaler Works

Scale Up Flow

Pods pending (no space)
↓
Cluster Autoscaler detects it
↓
Requests cloud provider (AWS/GCP/Azure)
↓
New node created
↓
Scheduler places pods

Scale Down Flow

Low usage nodes
↓
Autoscaler checks if pods can move elsewhere
↓
Drains node (moves pods)
↓
Deletes node

5. Key Condition for Scaling Up

A node is added ONLY when:

👉 Pods are **Pending due to insufficient resources**

NOT because CPU is high (that's HPA's job)

6. Important Concept — Unschedulable Pods

Example:

Pod requests:

cpu: 2 cores

Node available:

only 1 core free

👉 Pod becomes **Pending**

👉 Cluster Autoscaler says:

“We need a bigger node or more nodes”

7. Requirements

⚠️ Cluster Autoscaler **does NOT work in bare minikube like real cloud**

You need:

- AWS (EKS)
- GCP (GKE)
- Azure (AKS)

Because it needs:

👉 Cloud API to create/delete nodes

8. Configuration (Conceptual)

In cloud, you define:

Node Group:

min nodes: 1

max nodes: 10

Autoscaler works within this range.

9. Example Scenario (Real Thinking)

Step 1 — Normal load

- 2 nodes
- 4 pods running

Step 2 — Traffic spike

- HPA scales → 10 pods
- Only 6 pods fit

👉 4 pods → Pending

Step 3 — Autoscaler reacts

Pending pods detected

→ Add new node

→ Pods scheduled

Step 4 — Traffic drops

- Pods reduce to 3
- One node becomes mostly empty

👉 Autoscaler removes that node

10. Interaction Between HPA and Node Autoscaler

This is VERY IMPORTANT 🔥

HPA → decides how many pods needed

Cluster Autoscaler → ensures nodes exist to run them

Flow:

Traffic ↑

↓

HPA → increases pods

↓

Pods Pending?

↓ YES

Cluster Autoscaler → adds nodes

11. Key Differences

Feature	HPA	Node Autoscaler
Scales	Pods	Nodes
Trigger	CPU/memory/metrics	Pending pods
Works in minikube	✅	❌ (limited)
Cloud dependency	❌	✅

12. Common Mistakes

- ❌ Thinking autoscaler reacts to CPU → NO
 - ❌ Not setting resource requests → scheduler can't decide
 - ❌ Very high requests → unnecessary node scaling
 - ❌ No max node limit → cost explosion 💸
-

13. Mental Model (Simple)

Pods need space → Nodes provide space

- HPA → “We need more workers”
 - Autoscaler → “We need a bigger office”
-

14. Full System View (This is what pros think)

User Traffic ↑



HPA scales pods ↑



Cluster full?



Cluster Autoscaler adds nodes



Pods get scheduled

15. Final Insight (Important)

If your system is breaking, always ask:

1. Are pods scaling? → HPA issue
2. Are pods pending? → Node autoscaler issue
3. Are nodes full? → Resource request issue